

```

        android:paddingBottom="@dimen/activity_vertical_margin"
        android:paddingLeft="@dimen/activity_horizontal_margin"
        android:paddingRight="@dimen/activity_horizontal_margin"
        android:paddingTop="@dimen/activity_vertical_margin">
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@{user.name}"
            android:textAppearance="?android:attr/textAppearanceLarge"
            android:textSize="20sp"
            tools:text="Test Data"/>
    </LinearLayout>
</layout>

```

The model object has to be specified inside the data element. This shall append a setter to the “user” object on the binding class which is generated automatically. The binding classes are generated automatically based on the layout, hence they are named according to the name of the layout file. You can bind the Values to attributes by specifying an expression generic to the Data Binding Library. The expressions are Java statements with some aiding operators to make things simpler.

In order to access the binding within an activity, you will have to inflate the layout using method `setContentView()` of the `DataBindingUtil`. The `setContentView()` method would return a `ViewDataBinding` instance generic to the inflated layout .

```

@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
    FragmentSimpleBinding binding = DataBindingUtil.
    inflate(inflater, R.layout.fragment_simple_binding, container, false);

    user = new SimpleBindingUser();
    user.setFirstName(getString(R.string.app_name));
    binding.setUser(user);

    return binding.getRoot();
}

```

Here, the `onCreateView`, which is obtained from the `ViewBinding` instance returns the root view of the inflated layout. The `setUser` method